

분석할 수 없는 데이터에서 분석할 수 있는 데이터까지

구 서비스 DB에서 발견한 구조적 결함을 출발점으로, 결함을 담은 스키마를 설계하고, 그 스키마를 실제 트래픽으로 검증할 웹앱을 올리고, 데이터를 BigQuery까지 흘린 9일간의 기록.

기준일	2026년 8월 6일
기간	2026-07-29 ~ 2026-08-06 (9일) · 커밋 115개
산출물	스키마 40테이블 342컬럼 · 웹앱 배포본 20화면 · 적재 파이프라인 · 합성 데이터 1억 2,370만 행
대상	이 프로젝트를 처음 접하는 데이터 실무자

00 한 장 요약

이 프로젝트의 목적은 서비스 운영이 아니라 분석 가능한 데이터를 만드는 구조를 세우는 것이다. 웹앱과 합성 데이터는 그 구조를 검증하고 채우는 수단이다.

40 TABLES · 342 COLS	1.24억 ROWS IN BIGQUERY	201 RLS TESTS PASSED	83 DECISION NODES
-------------------------	---------------------------	-------------------------	----------------------

다섯 문장 요약

- 출발점은 구 서비스 DB 분석이다. 가입자 67.7만 명, 14개월치 데이터를 분석하는 과정에서 답을 낼 수 없는 질문이 반복해서 나왔고, 원인은 데이터가 부족해서가 아니라 스키마가 그 질문을 기록하지 않는 형태였기 때문이다.
- 결함을 담은 스키마를 새로 설계했다. 40테이블 342컬럼. 탈퇴에 유저 식별자를 넣고, 하트 잔액을 원장으로 설명 가능하게 하고, 관계를 전부 FK로 선언했다.
- 웹앱은 그 스키마를 실제로 돌려보기 위한 장치다. 실유저가 접속해 투표하면 스키마·권한·적재 경로가 동시에 시험된다. 20개 화면(W0~W19)을 3일에 붙였고, 이 과정에서 설계 문서만으로는 드러나지 않던 결함이 실제로 나왔다.
- 분석 물량은 합성 데이터로 만든다. 실유저는 23명이라 분포를 볼 수 없다. 2만 명·12개월·1억 2,370만 행을 생성했고, 판본을 다섯 번 거치며 검증했다.
- 문서는 위키로 재편했다. 코드가 매 커밋 바뀌므로 문서는 조용히 낡는다. 결정을 노드로 쪼개고, 문서의 주장을 살아 있는 DB와 대조하는 검사를 코드로 두었다.

이 문서의 구성

01-02	왜 시작했나. 구 서비스에서 발견한 결함과, 이 프로젝트가 무엇이고 무엇이 아닌지
03	전체 연표. 9일을 다섯 국면으로
04-07	산출물 넷. 스키마 · 웹앱 · 파이프라인 · 합성 데이터
08	문서 체계. 왜 위키로 재편했고 무엇을 얻었나
09	관측된 실패. 조용히 틀렸던 것들과 그것을 막은 장치
10-11	현재 상태와 남은 일
12	용어

01 출발점 — 구 서비스 DB에서 무엇이 막혔나

선행 작업은 기존 학교 소셜 투표 서비스(`final` · `hackle` 스키마)의 DB 분석이었다. 가입자 677,085명, 14개월 치, 리포트 9종으로 정리했고 원본은 `raw/legacy-analysis/`에 있다.

1.1 분석 결과 자체는 명확했다

서비스의 실태는 수치로 분명하게 드러났다. 아래는 그중 핵심 다섯이다.

발견	수치
서비스가 사실상 10개 학교에서만 작동	5,951개교 중 투표가 발생한 곳은 학생 수 상위 10개교뿐. 11위부터는 예외 없이 0건. 해당 10개교 소속은 전체 가입자의 0.75%
매출과 핵심 기능의 분리	결제유저 59,192명 중 99.3%가 애초에 투표가 불가능한 학교 소속. 전체 매출 약 2억 원의 99.36%가 투표 비활성 학교에서 발생
리텐션 붕괴	가입 후 1주 내 이탈 63.9%, 30일 이상 활동 유지 4.1%. 대량가입 코호트(전체의 93.9%)의 30일 잔존율은 3.9%
병목은 투표가 아니라 결과 공개	질문 노출 → 투표 완료 전환율 96.3%. 그러나 결과 알람을 열람한 사람의 83.5%가 끝내 답변을 공개하지 않음
신고와 제재의 단절	피신고 10회 이상 116명 중 제재된 사람 0명. 253회 신고된 계정도 정상 상태 유지

1.2 그런데 답을 낼 수 없는 질문이 반복해서 나왔다

분석이 막힌 지점들은 데이터 양의 문제가 아니었다. 스키마가 그 질문을 기록하지 않는 형태였다. 이 프로젝트의 실제 출발점은 1.1이 아니라 아래 셋이다.

구조적 결함	결과
탈퇴 기록에 유저 식별자가 없다 <code>accounts_userwithdraw</code> · 70,764건 · 전체의 10.5%	<code>reason</code> 과 <code>created_at</code> 만 있어 누가 탈퇴했는지 특정할 수 없다. 탈퇴 사유를 유저 속성(가입 경로 · 활동량 · 학교 · 결제 여부)과 교차분석하는 것이 원천적으로 불가능하다. 이탈 원인 분석이라는 가장 기본적인 과제가 성립하지 않는다
하트 원장이 잔액을 설명하지 못한다	재화의 증감 이력과 현재 잔액이 서로 맞지 않는다. 재화 경제의 어느 지점에서 얼마가 들어오고 나가는지 재구성할 수 없다
관계 39개 중 FK 선언은 17개	참조 무결성이 애플리케이션 코드에만 존재한다. 고아 행이 실제로 존재하는지 매번 확인해야 하고, 조인 결과를 신뢰할 근거가 DB 안에 없다

이 문서 전체를 관통하는 판단. 위 셋은 운영 중에는 아무 문제도 일으키지 않는다. 서비스는 잘 돌아갔고, 오류도 나지 않았다. 결함은 분석을 시도하는 순간에만 드러난다. 그래서 이 프로젝트는 "분석할 수 있는 형태로 기록하는 것"을 스키마 설계의 1차 목표로 두었다.

1.3 결함이 던진 두 번째 문제 — 분석할 데이터가 없다

결함을 닫은 스키마를 설계하는 것만으로는 검증이 되지 않는다. 새 스키마가 실제로 질문에 답하는지 확인하려면 그 스키마 위에 쌓인 데이터가 있어야 한다. 구 서비스의 데이터는 옛 스키마 형태이므로 옮겨 쓸 수 없다. 여기서 두 갈래가 나온다.

- **실데이터** — 새 스키마 위에서 실제로 발생시킨다. 그러려면 사람이 쓸 수 있는 서비스가 있어야 한다. 이것이 웹앱을 만든 이유다(5장).
- **합성 데이터** — 지인 20~50명으로는 분포를 볼 수 없다. 분석 연습과 파이프라인 부하 시험에 필요한 물량은 생성한다(7장).

02 이 프로젝트는 무엇인가

목표를 오해하면 판단 기준이 흔들린다. 이 장은 무엇이 목표이고 무엇이 범위 밖인지를 명시한다.

2.1 목표

데이터 파이프라인과 분석이 목표다. 웹앱은 실패데이터를 만드는 수단이고, 지인 대상 클로즈드 테스트는 그 데이터를 얻는 방법이다. 기능 판단이 갈릴 때의 기준은 "제품에 유리한가"가 아니라 **"분석할 데이터가 남는가"**다.

이 기준이 실제로 작동한 사례가 있다. 이용자가 여러 학교로 흩어지는 것은 제품 관점에서는 손해다. 후보 풀이 얇아지기 때문이다. 그러나 분석 관점에서는 학교라는 차원이 생기는 이득이므로, 스코프를 낮추는 대신 후보를 다른 친구로 채우고 채운 수를 `vote_item.padded_count`에 기록하는 쪽을 택했다.

2.2 핵심 제약 — 개인정보를 받지 않는다

받지 않는 것	대신
이메일 · 비밀번호	익명 계정. 접속 즉시 자동 생성되며 로그인 절차 자체가 없다
전화번호	친구 추가는 초대 코드로. 웨이라 연락처 동기화가 불가능하기도 하다
실명	유저가 정하는 닉네임

유저가 입력하는 것은 **닉네임 · 성별 · 소속(학교 · 반)**이다. 성별은 받은 투표의 힌트로 판매하는 정보이므로 온보딩 필수 항목이다.

이 조합은 소규모 집단에서 개인 특정이 가능하다. 비개인정보라고 단정하지 않고 개인정보처리방침에 수집 항목으로 명시했다. 만 14세 미만 가입 금지는 자기신고로 갈음한다. 생년월일을 받지 않으므로 검증 수단이 없고, 받으면 그 자체가 개인정보 수집이 되기 때문이다.

2.3 이 프로젝트가 아닌 것

상용 서비스 출시	앱스토어 배포 · 인앱결제 · 사업자등록은 범위 밖
불특정 다수 공개	성인 지인 20~50명 대상 클로즈드 테스트만. 온보딩에서 학교를 자유롭게 고를 수 있고 익명 계정이 무제한이므로, "그 학교 사람인가"는 기술적으로 보증되지 않는다. 공개 확대 시 가장 먼저 검토할 항목이다
실결제 · 실광고	MVP에서 광고는 3초 대기 스텝, 하트 충전은 결제 없는 스텝이며 하루 한 번으로 제한한다. 스텝 결제는 <code>store_transaction_id</code> 의 MVP-STUB- 접두어로 구분하므로 매출 집계에서 반드시 제외한다
익명 게시판	신고 검토 인력이 없어 열지 않는다. 닉네임이 드러나는 자유게시판은 열었다. 글쓴이가 특정되면 "사고가 나도 책임을 물을 수 없다"는 전제가 바뀌기 때문이다

2.4 작업 조건

작업자	개발 경험 없음. 기획·데이터 해석 판단을 하며 코드는 직접 작성하지 않는다. 코드 작성과 디버깅은 전량 에이전트가 수행한다
비용	월 0~3만 원. 무료 tier를 벗어나는 선택지는 채택하지 않는다. Cloud Composer(월 40만 원대)를 제외하고 로컬 Docker Airflow를 쓰는 이유다
보안	RLS 없이는 실유저를 받지 않는다. 배포 전 침투·동작 시험 통과가 완료 조건이다
데이터 분리	실유저(Supabase)와 합성(로컬 Postgres)을 같은 DB에 섞지 않는다

03 전체 연표 — 9일, 다섯 국면

2026-07-29부터 08-06까지 커밋 115개. 국면마다 성격이 뚜렷이 다르다.

07-29	① 스키마를 먼저 세운다 구 서비스 분석에서 나온 결함을 담은 DDL을 작성하고(P0), 합성 데이터 생성기의 초판을 만들고(P1), 익명 인증에 맞춰 스키마를 조정했다(W0). 개인정보를 받지 않는다는 제약이 이 시점에 스키마 형태를 결정했다.
07-29 ~ 07-31	② 웹앱을 끝까지 밀어붙인다 W1 Supabase 구축과 RLS 침투 시험을 통과한 뒤 W2 앱 뼈대부터 W7 배포까지 하루 반 만에 도달했다. 이어 W8~W19를 이틀에 붙였다. 같은 구간에서 P3(NEIS 학교 데이터)와 P4(BigQuery 적재)가 함께 올라갔다.
07-31	③ 문서를 위키로 재편한다 단일 파일이던 결정 이력을 노드 56개로 분해하고, raw · docs · CLAUDE.md 3계층을 세우고, 문서의 주장을 살아 있는 DB와 대조하는 검사를 코드로 두었다. 같은 날 합성 데이터 v1(786만 행)을 전량 폐기했다. 마이그레이션이 반영된 뒤라 그 데이터로 분석하면 틀린 답이 나오기 때문이다.
08-03 ~ 08-05	④ 합성 데이터를 네 번 만들고 세 번 버린다 생성기가 40개 표를 전부 채우게 한 뒤 v2 → v3(폐기) → v4(확정) → 정밀 EDA에서 결함 발견 → v5로 이어졌다. 가장 길고 반복이 많은 구간이며, 이 프로젝트에서 가장 많은 결정 노드가 만들어진 구간이기도 하다.
08-06	⑤ 팀에 넘긴다 v5 전량을 BigQuery에 적재하고(파티션 한도에 한 번 막혔다), 팀원 4명의 접속 경로와 쿼리 규칙을 문서화하고, 접속 진단 노트북을 만들었다. 파이썬 환경을 .venv 하나로 통일하고, 합성 데이터 재현성을 실측해 배포 방식을 정했다.

국면별 성격

국면	주된 활동	이 국면이 남긴 것
① 스키마	설계	40테이블 DDL. 이후 모든 작업의 기준
② 웹앱	구현	실데이터 발생원. 설계로는 안 보이던 결함 노출
③ 위키	정리	결정의 이유가 남는 구조와 낡음 탐지 장치
④ 합성	검증 반복	분석 가능한 물량과 그 한계 목록
⑤ 인계	운영화	팀이 쓸 수 있는 상태

04 산출물 ① 스키마 — 결함을 어떻게 닫았나

40테이블 342컬럼, FK 75개. 구조의 진실은 DDL이며 문서는 정의를 복사하지 않는다. ERD는 손으로 그리지 않고 살아 있는 스키마에서 `db/erd.py` 가 뽑는다.

4.1 1장의 결함에 대한 대응

구 서비스의 결함	새 스키마
탈퇴에 유저 식별자 없음	<code>user_withdrawal</code> 이 유저를 참조한다. 탈퇴 사유를 활동량·소속·결제 이력과 교차할 수 있다
하트 원장이 잔액을 설명 못 함	<code>heart_transaction</code> 이 모든 증감을 기록하고 <code>heart_balance</code> 와 대조 가능하다. 정합성 검사 17종 중 다수가 이 대조다. 클라이언트에는 두 표 모두 쓰기 권한이 없고 RPC 함수로만 변경된다
관계 39개 중 FK 17개	관계를 전부 FK로 선언했다(75개). 고아 행이 존재할 수 없다

4.2 설계에서 함께 정한 것

끊어진 관계도 남긴다	친구를 끊어도 행을 지우지 않고 <code>ended_at</code> 을 찍는다. 관계의 생애가 기록되고, "통틀어 몇 명과 친구였는가" 같은 질문이 성립한다
컬럼 단위 은닉은 뷰로	RLS는 행 단위라 컬럼을 숨기지 못한다. <code>vote_received</code> 는 직접 접근을 막고 <code>my_vote_received</code> 뷰로만 노출한다. <code>voter_id</code> 는 하트를 받고 파는 유료 정보이기 때문이다
빈 표에도 이유를 붙인다	비어 있다는 사실 자체가 정보다. "기능은 살아 있는데 아무도 안 함"과 "화면·코드가 없음"을 구분해 <code>db/erd.py</code> 의 <code>EMPTY_REASON</code> 에 기록하고, 사유를 적지 않으면 ERD 화면이 "사유 미기재"로 드러난다

4.3 스키마를 바꿀 때의 제약

마이그레이션은 전부 다시 적용되는 것을 전제로 한다. 따라서 옛 마이그레이션도 새 스키마 위에서 실행 가능해야 한다. 이 전제는 표를 지우거나 이름을 바꾸는 순간 깨지지만, 옛 표가 실제로 존재하는 운영 DB에서는 드러나지 않는다.

2026-07-31에 두 건이 실제로 발생했다. 하나는 DDL이 더 이상 만들지 않는 표를 찾다가 실패했고, 다른 하나는 이미 존재하는 트리거를 다시 만들다가 실패했다. 둘 다 Supabase에서는 정상이었다. 대응으로 `db/replay_check.py` 를 두었다. 임시 DB를 만들어 DDL과 마이그레이션을 처음부터 전부 적용한 뒤 운영 스키마와 대조하고 삭제한다.

05 산출물 ② 웹앱 — 스키마와 파이프라인의 시험대

웹앱은 제품이 아니라 **검증 장치**다. 이 장은 왜 만들었는지를 먼저 밝히고, 무엇을 만들었으며, 실제로 무엇을 검증했는지를 정리한다.

5.1 왜 웹앱을 만들었나

스키마 설계와 합성 데이터만으로 끝낼 수도 있었다. 그렇게 하지 않은 이유는 셋이다.

스키마 검증	설계가 실제 사용 흐름을 감당하는지는 돌려봐야 안다. 온보딩·친구·투표·힌트 구매가 이어지는 동안 제약과 트리거가 의도대로 작동하는지, 정합성 검사가 실제 데이터에서 위반 0을 유지하는지는 사람이 쓰는 트래픽에서만 확인된다. 실제로 정합성 검사 3종이 설계와 어긋나 있던 것이 기능을 열면서 드러났다 (9장)
파이프라인 시험	증분 적재는 데이터가 계속 들어오는 상태에서만 시험된다. 워터마크가 전진하는지, 지운 행이 하류에 반영되는지, 실유저와 합성이 구분되는지를 확인하려면 실제로 증가하는 원천이 필요하다
실데이터 확보	분석 자체는 합성 데이터로 하더라도, 실유저의 로그가 흐르는 구조가 있어야 파이프라인이 의미를 갖는다. 구 서비스 분석에서 얻은 교훈이 새 스키마에서 실제로 해소되는지는 실데이터로만 확인할 수 있다

웹앱이 검증 장치라는 위치가 우선순위를 정한다. 화면의 완성도보다 데이터가 남는 형태가 우선이고, 결제·광고처럼 데이터 형태에 영향이 없는 부분은 스텝으로 대체했다. 대신 접속 로그는 반드시 쌓는다.

5.2 무엇을 만들었나

Next.js 16.2 / React 19.2 / TypeScript / Tailwind. Vercel 배포이며 `main` 푸시 시 자동 반영된다. 단계 20개를 3일에 붙였다.

단계	기능	데이터 관점의 의미
W0-W2	익명 인증 대응 스키마 · Supabase + RLS · 앱 뼈대	계정이 개인정보 없이 생성된다
W3	온보딩 · 초대 코드 발급	닉네임·성별·소속이 이때 한 번 수집된다
W4	친구(코드 교환, 5명 게이트)	소셜 그래프의 원천
W5	투표 — 후보 추출·서플·하트 적립	핵심 활동 로그, 스코프 3종
W6	받은 투표 · 힌트 구매	하트 소비처, 재화 경제의 반대편
W7	배포 · 초대 링크 · 개인정보처리방침	실데이터 유입 개시
W8-W16	학교 정보 — 급식표 · 학사일정	외부 데이터(NEIS) 결합 지점
W9	자유게시판(닉네임 노출) · 신고	투표 외 활동 축, 신고율 관측
W10	친구 추천(같은 학교)	추천 수락·거절 퍼널
W11-W13	프로필 수정 · 계정 삭제 · 하트 충전 스텝	탈퇴 사유가 유저와 함께 기록된다
W14-W15	선택형 힌트(5+1) · 1회성 답장	하트 지불 의사에 단계별 관측
W17-W18	운영자 테이블 제거 · 이름 정리	41 → 40테이블
W19	친구 끊기	관계 종료 시각 기록, 정합성 검사 3종 교정 계기

5.3 보안 원칙

실유저를 받는 서비스이므로 권한 설계를 먼저 확정했다. 원칙은 **읽기는 필요한 것만, 쓰기는 거의 열지 않는다**이다.

- **하트·투표 조작은 전부 RPC 함수로 처리한다.** 클라이언트가 원장에 INSERT 하거나 잔액을 UPDATE할 수 있으면 재화를 무한정 만들 수 있다.
- **가입도 수정도 RPC 하나뿐이다.** W1에서 프로필 컬럼에 UPDATE를 열어두었다가 W11에서 회수했다. 그 경로가 온보딩의 검증(길이 제한·성별 필수·선택 가능한 학급)을 전부 우회했기 때문이다.
- **id로 남을 지목할 수 있는 경로를 만들지 않는다.** `app_user.id`는 연속된 정수다. 친구 요청 INSERT를 열었더니 코드 없이 전체 가입자를 지목할 수 있었다. 친구 관련 쓰기는 전부 RPC로만 한다.
- **양방향으로 검증한다.** 전부 막아도 침투 시험은 통과하므로, 정상 동작 시험이 함께 있어야 의미가 있다. 현재 침투·동작 시험 201항목이며 배포 전 통과가 필수 조건이다.

기술로 막하지 않는 것도 명시했다. 게시판의 학교 경계는 14개 경로를 실제로 뚫어보고 전부 차단된 것을 확인해 시험에 포함했다. 그러나 **소속은 자기신고**이므로 "그 학교 사람인가"는 보증되지 않는다. 막을 수 있는 것과 막을 수 없는 것을 구분해 기록하는 것이 이 프로젝트의 보안 문서 방식이다.

5.4 웹앱이 실제로 검증한 것

스키마	실사용 흐름에서 정합성 17종 위반 0을 유지한다. 다만 기능이 늘면서 검사 쪽이 낡는 현상이 관측됐다(9장)
권한	RLS 201항목 통과. 동시에 이 시험이 재현하지 못하는 영역 이 있음을 확인했다. 세션 설정에 딸린 동작이 그렇다(9장)
파이프라인	실유저 데이터가 Supabase에서 BigQuery까지 증분으로 흐른다. 증분이 처음부터 흐르므로 소급 적재가 필요 없고, 이 때문에 클로즈드 테스트 초대를 미룰 이유가 사라졌다

06 산출물 ③ 파이프라인 — 저장소 셋이 갈라지고 만나는 곳

데이터는 세 곳에 있다. 각각 무엇이 들어 있고 어디서 합쳐지는지를 아는 것이 이 파이프라인을 이해하는 핵심이다.

Supabase

실유저 DB. 웹앱이 직접 쓴다. 가입자 23명 규모. 운영 중이며 접속은 Session pooler로만 한다.

로컬 Postgres

합성 데이터 DB(`pgtest`). v5 1억 2,370만 행. 실유저와 같은 DB에 섞지 않는다.

BigQuery

분석 DW. 위 둘이 여기서 만난다. 데이터셋 `raw` · 40개 표 · 1억 2,373만 행 · 8.74 GiB.

6.1 두 원천을 어떻게 구분하나

표식이 둘 있다. `_source` 는 어느 저장소에서 왔는지, `is_synthetic` 은 생성된 행인지를 나타낸다. 실유저만 보려면 `WHERE _source = 'supabase'` 를 쓴다.

조인할 때 이 표식을 빠뜨리면 결과가 부풀려진다. 두 원천의 id가 각각 1부터 시작하므로 사실상 전부 충돌한다. `vote_item` 과 `app_user` 를 `_source` 없이 조인하면 **14,286행이 늘어난다**. 실측한 값이며, `stg` 층이 아직 없어 분석자가 `raw` 를 직접 보는 현재 상태에서는 이 함정이 살아 있다. 6장의 규칙 중 가장 먼저 지켜야 할 항목이다.

6.2 단계별 상태

단계	내용	상태
P0	스키마 · DDL	완료 40테이블 342컬럼
P1	합성 데이터 생성	완료 v5 · 2만 명 · 12개월 · 1억 2,370만 행
P2	Postgres 적재	완료 순서 정의 40표. 시퀀스·위터마크 보정 자동 실행
P3	NEIS 수집	일부 학교·학급·급식·학사일정 완료. DAG화가 남음
P4	BigQuery 적재	완료 40테이블 · 행 수 대조 통과
P5	품질 검증	예정
P6	stg / mart	예정
P7	대시보드	예정

6.3 증분 적재에서 실제로 걸린 함정

증분 적재는 실패해도 오류를 내지 않는다. 조용히 적은 행을 올리거나 아무것도 올리지도 않고 정상 종료한다. 그래서 `verify_load.py` 로 행 수를 대조하는 절차를 고정했다. 지금까지 실제로 걸린 것은 넷이다.

<code>_source</code> 없는 조인	id 충돌로 행이 부풀려진다
컬럼 추가 후 위터마크 정지	새 컬럼이 채워져도 <code>updated_at</code> 이 움직이지 않으면 적재되지 않는다
지운 행의 유령	원천에서 삭제된 행이 DW에 남는다. <code>_deleted_at</code> 으로 표시하도록 고쳤다
과거 시각 백필	위터마크보다 이전 시각으로 들어온 행은 다음 증분에서 누락된다

6.4 대량 적재는 파티션 한도에 막힌다

파티션 테이블은 하루 4,000회의 파티션 수정만 허용하며, 적재 작업 하나가 N개 파티션에 걸치면 N회를 소비한다. 세는 단위가 작업이 아니라 작업 × 파티션이다. 저장 순서대로 데이터를 꺼내면 한 덩어리가 수십 일치에 흩어지므로 한도가 빠르게 찬다.

2026-08-06에 `vote_candidate` 가 2,620만 행 지점에서 실제로 막혔다. 해결은 추출 쿼리에 **파티션 컬럼 기준 정렬**을 넣는 것이었다. 한 덩어리가 하루치에 수렴해 작업당 1~2회로 떨어진다. 해당 컬럼에 인덱스가 있어 정렬 비용은 없고, 부수적으로 적재 속도가 초당 12,732행에서 23,215행으로 **1.8배** 빨라졌다.

07 산출물 ④ 합성 데이터 — 다섯 판본과 그 한계

실유저 23명으로는 분포·퍼널·코호트를 볼 수 없다. 분석 연습과 파이프라인 부하 시험에 필요한 물량을 생성한다. 판본을 다섯 번 거쳤고, 그 과정 자체가 이 프로젝트에서 가장 많은 검증을 수행한 구간이다.

7.1 판본 이력

판본	규모	결과
v1	786만 행	전량 폐기. 정리가 아니라 데이터가 틀려서다. 마이그레이션 반영 이후라 이 데이터로 분석하면 틀린 답이 나온다. 보관 가치가 0이 아니라 음수였다
v2	1억 8,826만 행	전면 감사에서 결함 3건 발견
v3	1억 8,918만 행	폐기. 확정 직전 세션 로그 결함이 나와 중단. 측정값은 기록으로 남겼다
v4	1억 8,922만 행	목표 지표 10개 달성으로 확정했으나, 정밀 EDA에서 결함 1건·경계 5건 확인
v5	1억 2,370만 행	확정판. 정합성 17종·이상치 9종 위반 0. v4에서 불가능하던 15종 중 9종이 열렸다

v5의 행 수가 v4보다 35% 적은 것은 줄어든 것이 아니라 없어야 했던 것이 빠진 결과다. v4에는 하루에 406.8시간을 투표한 유저-일이 존재했다. 24시간을 초과한 유저-일이 1,251건(67명)이었고, 한 계정은 하루 554세션·99.2시간을 기록했다. 정합성 17종과 이상치 9종은 이것을 잡지 못한다. "하루에 몇 번까지가 사람인가"는 스키마 제약으로 표현되지 않기 때문이다.

7.2 상한을 개수로 걸었다가 되돌린 사례

위 결함의 대응으로 하루 세션 수에 상한을 걸었더니 헤비유저가 통째로 사라져 행이 절반 가까이(9,509만) 줄고 지목 쓸림이 39%로 떨어졌다. 요구는 "물리적으로 불가능한 활동을 없애라"였지 "상한을 정하라"가 아니었으므로, **시간 예산**(투표 6시간/일)으로 바꿨다. 1억 2,370만 행·쓸림 49.8%로 회복했다.

같은 작업에서 인기도 파라미터를 0.45에서 1.05로 올렸다. v4의 쓸림 48.8%가 인기도가 아니라 불가능한 활동 꼬리에 의해 만들어지고 있었기 때문이다.

7.3 이 데이터로 할 수 없는 것 7가지

합성 데이터의 한계를 명시하는 것은 데이터를 만드는 일의 일부다. 설계에 차이를 넣지 않은 측은 분석해도 차이가 나오지 않으며, 그것을 모르고 검정하면 반드시 기각된다.

불가능한 분석	이유
질문별 인기 · A/B	출제량 차이가 설계된 것이 아니라 표본 노이즈
후보 슬롯 위치 편향	변동계수 0.0039
문항 위치별 응답 품질	변동계수 0.0002
성별 상호작용(동성 선호)	지목 비율이 인구 비중과 동일
게시판 카테고리별 차이	변동계수 0.0046
앱 버전 · 플랫폼 · 기기 단위	버전이 매달 33%로 고정. 기기 1개가 플랫폼 3개를 오간다
광고 네트워크 · 시청시간 분포	네트워크가 하나뿐이고 시청시간이 균등분포

주의해서 다뤄야 할 것 셋

- **중복 로그 0.4%를 걸러내지 않으면 체류시간이 부풀려진다.** 의도적으로 넣은 노이즈이며, 정제 연습을 위한 것이다.
- **시각은 UTC로 저장된다.** KST로 변환하지 않으면 시간대 봉우리가 9시간 밀린다. 밤 22시가 낮 13시로 보인다.
- **제재 코호트는 시각으로 잘라야 한다.** 34명이 제재 이후에도 투표한다. 탈퇴만 종점으로 처리했기 때문이다.

7.4 재현성 — 덤프를 나누지 않는 이유

팀원이 로컬에서 같은 데이터를 쓰려면 11GB 덤프를 배포해야 하는지가 문제였다. 답은 아니었다. 같은 시드와 같은 설정으로 다시 만들면 같은 결과가 나온다는 주장을 전체 규모로 재생성해 **CSV 33개 전부 sha256 일치**로 확인했다. 생성 시간은 **14분 19초**다. 이전 문서의 "1~2시간"은 실측 없이 적힌 값이었다.

08 문서 체계 — 왜 위키로 재편했나

문서 재편은 정리 취향의 문제가 아니라 관측된 문제에 대한 대응이다. 이 장은 그 명분과 실측된 이득, 그리고 남은 비용을 함께 기록한다.

8.1 문제 셋

문서가 조용히 늙는다	이 저장소는 코드도 진실이고 코드는 매 커밋 바뀐다. 문서의 주장이 현실과 어긋나도 오류가 나지 않으므로 발견되지 않는다. 실제로 점검했을 때 낡은 주장이 9곳 있었고, "비어 있는 테이블" 목록은 손으로 관리되던 탓에 내용이 틀려 있었다
결정의 이유가 커밋 메시지에만 남는다	합성 데이터 작업 구간에서 결정 노드를 아홉 번 연속으로 건너뛰었고, 근거가 커밋 메시지와 설정 파일 주석에만 남았다. 몇 주 뒤 같은 값을 다시 조정할 때 왜 그렇게 정했는지 복원할 수 없다
단일 파일의 읽기 비용	결정 이력이 한 파일 24,946토큰이었다. 결정 하나를 확인하려고 전체를 읽으면 낭비율이 98.2%다

8.2 3계층 구조

층	무엇	고치는 주체
1	raw/ 원본	사람만 고친다. 회의록·구 서비스 분석·외부 스펙. 나중에 결정이 바뀌어도 원본은 그대로 둔다. 바뀐 것은 새 결정으로 적는다. 그래야 "언제 왜 바뀌었나"가 남는다
2	docs/ 위키	에이전트가 소유한다. 결정 83개 · 표 페이지 40장 · 운영 문서 7개
3	CLAUDE.md 규약	사람과 에이전트가 함께 다듬는다

8.4 낚음을 잡는 장치

`db/doc_lint.py` 가 문서의 주장을 살아 있는 DB·파일시스템과 대조한다. 아홉 가지를 검사하며, 뒤의 셋은 실제로 겪은 사고를 잡으려고 추가한 것이다.

숫자 · 사라진 이름 · 끊어진 문서와 현실의 기본 대조
링크 · 없는 스크립트 · 낡은
색인 · 미반영 회의록

없는 질문 번호	질문 번호는 위치 기반 식별자다. 원본에서 질문을 지웠더니 그것을 인용한 문서 4개가 조용히 깨져 있었다
결정 없이 쌓인 코드	마지막 결정 노드 이후 코드 커밋이 8개를 넘으면 알린다
색인에서 빠진 결정 노드	색인에 없는 노드는 사실상 없는 노드다. 특정 그룹의 노드 3개가 절 자체가 색인에 없어 통째로 누락돼 있었다

검사 결과는 **고쳐야 하는 것과 사람이 판단할 것**으로 나뉜다. 숫자와 이름은 이력 서술과 기계적으로 구별되지 않으므로 알리기만 한다. 전부 실패로 처리하면 항상 빨간불이 되고, 그러면 검사 자체를 보지 않게 된다.

8.5 되돌릴 수 있게 해두었다

노드 분할이 손해로 판명될 가능성에 대비해 판단 기준과 복구 경로를 함께 두었다. 기준은 둘이다. 같은 질문에 토큰을 더 쓰거나, 관련 결정을 놓쳐 답이 나빠지거나.

복구는 `git revert` 로 하지 않는다. 되돌리는 커밋 이후에 작성된 결정이 함께 사라지기 때문이다.

`db/wiki_merge.py` 가 현재 존재하는 노드 전부를 읽어 단일 파일로 합치므로 새로 쓴 결정도 보존된다. 옛 원본과 대조해 본문 손실 0을 확인했다.

09 관측된 실패와 대응

이 프로젝트에서 발생한 문제는 대부분 오류를 내지 않는 종류였다. 공통 대응은 같다. 사고가 난 지점을 검사 코드로 고정한다.

9.1 시험이 통과해도 앱이 죽을 수 있다

2026-07-30, RLS 시험 128항목이 전부 통과한 상태에서 실제 투표가 실패했다. 원인은 브라우저가 거치는 역할에 `safeupdate`가 걸려 있어 WHERE 없는 DELETE가 임시 테이블에서도 차단된다는 것이었다. 시험 코드는 관리자 계정으로 접속한 뒤 역할만 전환했는데, 해당 설정은 세션이 열릴 때 적용되므로 재현되지 않았다.

대응으로 시험이 소스를 직접 검사하도록 했다. RLS SQL에 WHERE 없는 DELETE/UPDATE가 있으면 실패 처리한다.

교훈 — 역할을 바꾸는 것과 그 역할로 접속하는 것은 다르다. 세션 설정에 딸린 동작은 역할 전환으로 재현되지 않는다.

9.2 정합성 검사가 기능보다 낫는다

친구 끊기(W19)를 열자 평소에 드러나지 않던 검사 오류 3건이 한꺼번에 발생했다. 셋 다 데이터가 아니라 **검사 쪽이 틀린** 경우였다.

게이트 위반	"현재 친구 5명 미만인데 해금됨"으로 물었다. 설계는 해금 시각을 한 번 찍으면 유지한다. 친구가 줄어드는 경로가 없던 시절에는 드러나지 않았다. "통틀어 5명을 맺어본 적이 있는가"로 고쳤다
원장 없는 힌트 구매	하트 0으로 열리는 광고 무료 힌트를 위반으로 잡았다. 원장은 하트의 이동을 적는 곳이지 힌트를 연 사실을 적는 곳이 아니다
친구 수 불일치	끊긴 관계까지 세고 있었다

반대로 **일부러 고치지 않은** 검사가 하나 있다. "투표 당시 친구였는가"를 묻는 검사에 현재 관계 조건을 붙이면, 친구를 끊는 순간 그 사람이 등장한 과거 투표가 전부 위반으로 잡힌다.

교훈 — 기능을 열 때 그 기능이 건드리는 검사를 함께 읽는다. 검사는 작성 시점의 세계를 알고 있고, 그 세계는 기능이 늘 때마다 바뀐다.

9.3 설정에 값이 있어도 코드가 읽지 않으면 없는 것이다

합성 데이터 생성기에서 설정 파일에 값이 존재하는데 코드가 그 절을 읽지 않는 문제가 사흘 동안 **아홉 번** 발생했다. 오류가 나지 않고 데이터도 전부 생성되므로, 완성된 데이터를 분석해야만 드러난다. 예를 들어 주말 물량 배수를 읽지 않아 요일별 총량의 변동계수가 0.024에 머물렀고, 그 결과 요일 효과 분석 자체가 불가능했다.

대응으로 설정 로딩 방식을 뒤집어 구조적으로 막았다. 다만 같은 작업 구간에서 읽히지 않는 설정을 또 만들었다. 구조적 방지책이 완전하지 않다는 사실도 함께 기록했다.

9.4 손으로 관리하는 목록은 낡는다

"비어 있는 테이블" 목록을 문서에 손으로 적어두었더니 내용이 틀어졌다. 3행이 쌓인 표를 비었다고 적어두고, 정작 비어 있는 표는 목록에서 빠져 있었다. 대응으로 목록을 `db/erd.py` 가 살아 있는 DB에서 세도록 옮겼고, 사유를 코드에 적게 했다.

9.5 한도에 막혔을 때 원인을 추정하지 않는다

파티션 한도 사고(6.4)에서는 대응 자체보다 판단 과정에 오류가 셋 있었다.

- 다른 한도를 원인으로 지목했고, 그에 맞춰 조정한 파라미터는 실제로는 무관했다.
- 적재 속도가 느려지는 것을 정상 동작으로 판단했다. 실제로는 한도가 차오르는 신호였다.
- 한도 초기화 시각을 추정해 "오후까지 기다려야 한다"고 판단했으나, 시도해보니 이미 풀려 있었다. 추정할 것이 아니라 시도해서 확인할 일이었다.

세 건 모두 정정 기록을 남겼다. 틀린 판단을 지우지 않고 정정으로 남기는 것이 이 저장소의 문서 방식이다.

9.6 공통 패턴

이 프로젝트의 사고는 전부 "조용히 틀리는" 종류였고, 대응은 전부 사후에 만들어졌다. `safeupdate` 소스 검사 · `replay_check.py` · `doc_lint.py` 의 뒤 세 검사 · `verify_load.py` · 노트북의 자격증명 경고가 모두 그렇다. 검사 장치는 설계로 미리 갖춰진 것이 아니라 사고 목록의 함수다.

10 현재 상태

2026년 8월 6일 기준. 앱은 배포돼 있고 실데이터가 BigQuery까지 흐른다.

20	5 / 8	8.74	14:19
WEB STEPS DONE	PIPELINE STAGES	GIB IN BIGQUERY	SYNTH REGEN TIME

10.1 완료된 것

스키마	40테이블 342컬럼 · FK 75 · 정합성 검사 17종 · 이상치 검사 9종
웹앱	W0~W19 완료. Vercel 배포 · RLS 침투·동작 시험 201항목 통과
파이프라인	P0·P1·P2·P4 완료. 실유저 데이터가 증분으로 BigQuery까지 도달
합성 데이터	v5 확정 · 2만 명 · 12개월 · 1억 2,370만 행 · 전량 BigQuery 적재 · 40개 표 대조 일치
문서	결정 83개 · 표 페이지 40장 · 운영 문서 7개 · PDF 리포트 다수 · lint 통과
팀 접근	팀원 4명 BigQuery 권한 · 접속 문서 · 진단 노트북. 키 파일은 배포하지 않는다

10.2 저장 용량

BigQuery raw 데이터셋은 8.74 GiB이며 무료 저장 한도 10 GiB 안에 있다. **여유가 1.26 GiB뿐**이므로 다음 대량 적재 전에 확인이 필요하다.

10.3 미완인 것

P3 NEIS	학교·학급·급식·학사일정 수집은 완료. DAG화가 남았다
W8 학교 정보	급식·학사일정은 화면이 있다. 시간표·공지 화면이 없다
화면이 없는 표 6개	block_record · question_request · sanction · timetable · school_notice · school_notice_read. 특히 신고가 처리 대기 상태로 고인다. 검토·제재 경로가 없다
클로즈드 테스트	초대를 아직 보내지 않았다. 증분이 처음부터 흐르므로 소급 적재는 필요 없다

10.4 비어 있는 표 8개의 해석

실서비스 기준으로 아직 한 줄도 쌓이지 않은 표가 8개다. 두 갈래로 나뉘며, 이 구분이 정보다.

기능은 살아 있으나 발생 0	추천 거절 · 탈퇴. 결함이 아니라 실측 결과다
화면·코드가 없음	위 10.3의 6개 표

11 남은 일

우선순위 순으로 정리한다. 1번은 바로 뒤에 착수할 작업이다.

1 · 다음

합성 데이터 분석

지금까지는 데이터를 만드는 작업이었고, 이제 그 데이터로 분석하는 단계다. BigQuery의 1억 2,370만행을 대상으로 리텐션·퍼널·코호트·재화 경제·소셜 그래프를 본다. 착수 전에 확인할 것이 둘 있다. ① 검수 리포트 13장의 "할 수 있는 것 / 주의 / 없는 것"을 먼저 읽는다. 설계에 차이를 넣지 않은 측은 검정하면 반드시 기각된다. ② `_source` 없는 조인과 UTC/KST 변환은 결과를 조용히 틀리게 만든다.

2 · P5

품질 검증

적재된 데이터에 대한 상시 검증 체계. 지금은 적재 직후 행 수 대조를 수동으로 실행한다. 이것을 규칙화하고 신선도·중복·널 비율·참조 무결성을 포함한 점검으로 확장한다. 검증 결과 자체가 기록으로 남아야 한다.

3 · P6

stg / mart 층

현재 분석자는 `raw` 를 직접 본다. 그래서 id 충돌 함정이 각 분석자에게 그대로 노출된다. `stg` 층에서 원천 구분·타입·시간대를 정리하면 이 함정이 구조적으로 사라진다. 그 위에 자주 쓰는 집계를 `mart` 로 고정한다.

4 · P7

대시보드와 일일 통계 자동화

Airflow DAG로 일 단위 집계를 돌리고 결과를 대시보드와 슬랙 일일 리포트로 내보낸다. 가입·활성·투표·하트·신고의 전일 수치와 이상 징후를 매일 아침 채널에 남기는 형태다. 이 단계가 붙으면 파이프라인이 "돌려야 결과가 나오는 것"에서 "돌고 있는 것"으로 바뀐다.

5 · P3

NEIS 수집의 DAG화

학교·학급·급식·학사일정은 현재 수동 스크립트로 받는다. 급식과 학사일정은 주기적으로 갱신돼야 하는 데이터이므로 스케줄에 올린다. 학사일정은 받은 기간을 지우고 다시 넣는 방식이라 먹등성이 이미 확보돼 있다.

6 · 웹앱

남은 화면

시간표·공지 화면(W8 잔여)과 신고 처리 경로. 신고가 대기 상태로 고이는 현재 구조는 구 서비스에서 지적했던 "신고와 제재의 단절"과 같은 형태다. 1:1 익명 메시지·채팅방처럼 자유 텍스트가 사람에게 직접 가는 기능은 차단·신고 화면이 먼저 있어야 검토한다.

7 · 운영

클로즈드 테스트 초대

성인 지인 20~50명. 실데이터의 절대량이 늘어나 실유저 기반 분석이 의미를 갖는다. 저장 여유가 1.26 GiB뿐이라는 점과 함께 판단한다.

착수 순서에 대한 판단

1번(분석)을 3번(stg/mart)보다 앞에 두었다. 층을 먼저 쌓는 것이 이론적으로는 깔끔하지만, **어떤 집계가 실제로 반복해서 필요한지는 분석을 해봐야 알 수 있다**. 먼저 분석하면서 반복되는 쿼리를 확인한 뒤 그것을 `mart` 로 고정하는 편이 추측으로 층을 설계하는 것보다 낫다. 다만 id 충돌 함정은 그동안 각 분석자가 직접 지켜야 하므로, 규칙을 문서로 배포한 현재 상태를 유지한다.

12 용어

이 프로젝트에서 특정한 의미로 쓰이는 말만 정리한다.

서비스

하트	앱 내 재화. 가입 지급·투표 적립·광고 시청으로 얻고, 힌트 구매·답장으로 쓴다. 충전은 MVP에서 스텝이다
힌트	받은 투표에서 "누가 나를 뽑았는지"에 대한 단서. 하트를 지불해 단계적으로 연다. 이 정보가 유료라는 점이 컬럼 단위 은닉의 이유다
스코프	투표 질문의 후보 범위. CLASS · SCHOOL · GLOBAL 세 종류
게이트 / 해금	친구 5명 이상이면 투표가 열리는 조건. 해금 시각은 한 번 찍히면 유지된다
초대 코드	친구 추가 수단. 전화번호를 받지 않으므로 이것이 유일한 기본 경로다
패딩	후보가 부족할 때 스코프를 낮추지 않고 다른 친구로 채우는 것. 채운 수를 기록하고 화면에도 밝힌다
스텝	실제 동작 없이 형태만 갖춘 구현. 광고는 3초 대기, 하트 충전은 결제 없이 즉시 지급하되 하루 한 번으로 제한한다

데이터

RLS	Row Level Security. 행 단위 접근 제어. 컬럼은 숨기지 못하므로 컬럼 은닉이 필요하면 뷰를 쓴다
RPC	DB에 정의한 함수. 클라이언트에 직접 쓰기를 열지 않고 이 함수로만 변경을 허용한다
워터마크	증분 적재의 기준 시각(<code>updated_at</code>). 이 값이 전진하지 않으면 적재가 조용히 멈춘다
<code>_source</code>	BigQuery 행이 어느 원천에서 왔는지의 표식. 조인 시 반드시 조건에 넣는다
<code>is_synthetic</code>	생성된 행인지의 표식. <code>_source</code> 와 역할이 다르다
페르소나	합성 데이터 생성 시 유저에게 부여한 행동 유형. 정답지에 해당하므로 분석자에게 제공하지 않는다
이상치 검사	정합성 검사와 별개로, 값의 분포가 현실적인지를 보는 검사 9종

문서

ingest	원본을 읽어 위키에 반영하는 절차. 결정과 논의를 가르고, 뒤집힌 결정에 표시를 달고, 생성물을 다시 뽑는다
query	물음에 답하는 절차. 이름을 보고 골라 읽으며 매칭된 노드를 전부 읽지 않는다
lint	문서의 주장을 현실과 대조하는 절차
superseded	뒤집힌 결정에 붙는 표시. 옛 결정을 지우지 않고 대체 관계를 남긴다
생성물	스크립트가 만드는 문서. 손으로 고치면 다음 생성 때 사라지므로 원본 쪽을 고친다

참고 문서

"이 값이 왜 이렇게 정해졌나" 합성 데이터 결정 이력 리포트 · `docs/decisions/`

"이 데이터 써도 되나" EDA 확정판 리포트 13장

"실데이터와 합성이 어떻게 다른가" 파이프라인 전체 그림 리포트

"이 표는 무엇이고 무엇이 얹혀 있나" `docs/tables/<표>.md`

"이 작업을 어떻게 하나" `docs/ops/`

ping-v2 종합 보고서 · 기준일 2026-08-06 · 커밋 115개 기준
원본은 `docs/PROJECT-REPORT.html` 이며 PDF는 산출물이다. 고칠 때는 HTML을 고치고 다시 뽑는다.